

基于 ARM NEON 的 FFT 性能优化

鲲鹏平台研讨报告

FFT-GRP3

2026 年 5 月 12 日

FFT 算法介绍

离散傅里叶变换 (DFT)

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2\pi i k n / N}, \quad O(N^2)$$

快速傅里叶变换 (FFT) – Cooley-Tukey

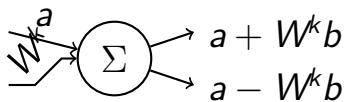
- 利用旋转因子 $W_N^k = e^{-2\pi i k / N}$ 的对称性和周期性
- 分治: N 点 DFT $\rightarrow$ 两个 $N/2$ 点 DFT
- 复杂度从 $O(N^2)$ 降至 $O(N \log N)$

蝶形运算

基-2 时间抽取蝶形公式：

$$X'[j] = X[j] + W_N^k \cdot X[j + N/2]$$

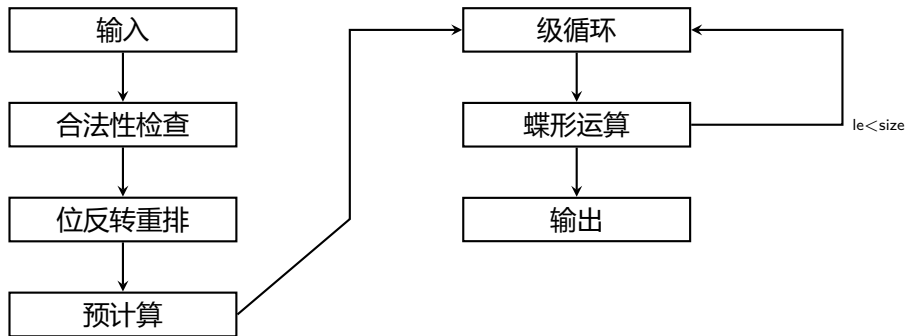
$$X'[j + N/2] = X[j] - W_N^k \cdot X[j + N/2]$$



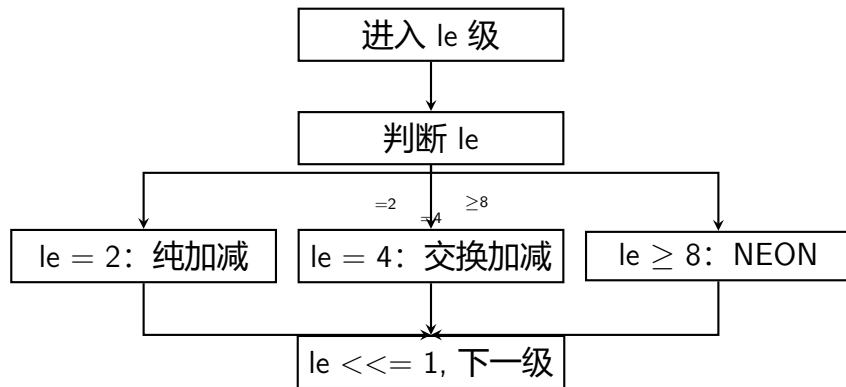
FFT 执行流程：

- 1 位反转重排输入序列
- 2 自底向上 $\log_2 N$ 级蝶形运算
- 3 每级 $N/2$ 个蝶形单元
- 4 每级内部不同蝶形组完全独立

程序主流程图



蝶形级循环细节



研讨要求 1: 接口命名规范

要求: 按功能 _ 平台 _IO 方式命名, 返回 RetCode_t

函数	功能	说明
fft_linux_iopointer	用户提供 I/O	input/output 可 cast 为结构体
fft_linux_ioself_profiling	自生成 + 计时	打印时间/误差/前 8 频点
fft_linux_ioself	自生成	不计时不打印
fft_linux_iofile	文件 I/O	读取.txt 做 FFT, 结果写回文件

结构体定义:

```
typedef struct {  
    float *real;  
    float *imag;  
} FftIO;
```

研讨要求 2: 时间函数

要求: 提供 `timestamp()` 和 `timestamp_diff()`

timestamp.h

```
typedef struct timespec
    timestamp_t;

timestamp_t timestamp(void);

int64_t timestamp_diff(
    timestamp_t start,
    timestamp_t end);
```

实现 (timestamp.c)

- 系统调用: `clock_gettime`
- 时钟源:
`CLOCK_MONOTONIC`
- 精度: 纳秒级
- 差值返回 `int64_t`

研讨要求 3：测试主函数

要求：循环不同规模采样，每次不受上次影响

```
int sizes[] = {
    256, 512, 1024, 2048, 4096, 8192,
    16384, 32768, 65536, 1<<17, 1<<18,
    1<<19, 1<<20
};
for (int i = 0; i < count; i++)
    fft_linux_ioself_profiling(sizes[i]);
```

采样独立性保障：

- 无全局变量：旋转因子表局部分配
- Twiddle Cache 按 size 更新，不跨次残留
- 每次调用独立分配/释放内存

研讨要求 4：文件结构

要求： 6 个源文件

文件	内容
fft_linux.h	公开 API 声明 (4 接口)
fft_linux.c	FFT 核心实现 + 5 个独立编译开关
fft_linux_test.c	性能测试主函数
fft_demo.c	示例程序 (自定义输入)
timestamp.h/c	纳秒级时间函数
input.txt	示例输入数据文件
readme.txt	编译执行说明

系统结构层次优化总结 (1/2)

硬件特性	对应优化
ARMv8 NEON SIMD 单元	4 路 SIMD 向量化蝶形 IFFT 共轭/缩放向量化 直接 vld1q_f32 加载 twiddle
Cache 层次	float 化存储 (原 double, 减半) SoA 布局: real/imag 分离连续访存 Twiddle Cache 复用
多核架构	OpenMP parallel for schedule(static) 均匀分配 size > 4096 启用

系统结构层次优化总结 (2/2)

硬件特性	对应优化
指令流水线	le=2/le=4 蝶形手写展开 消除循环分支 & 函数调用 减少分支预测失败
内存带宽	旋转因子迭代递推 $\cos/\sin \rightarrow 4 \text{ mul} + 2 \text{ add}$ 降低三角函数指令开销
NEON 对齐	posix_memalign(16) 保证向量访存稳定性

核心里念： 算法结构匹配硬件特性 —
计算密集 $\rightarrow$ SIMD，并行 $\rightarrow$ 多核，访存 $\rightarrow$ 优化布局

性能测试结果

平台: 鲲鹏 ARMv8 **编译:** -O3 -march=armv8-a -fopenmp

Size	Time (ms)	Max Error
256	0.0307	0.044922
512	0.0126	0.167969
1024	0.0276	0.246094
2048	0.0553	0.339844
4096	0.1271	1.572266
8192	0.3494	2.173828

验证: FFT $\rightarrow$ IFFT $\rightarrow$ 与原始输入对比

Max Error 为 float 累积误差, 相对误差 $< 0.001\%$

总结

优化覆盖层次：

- 算法层：旋转因子递推、蝶形特化
- 数据层：SoA、float 化、16B 对齐
- 并行层：OpenMP 多核
- 指令层：NEON SIMD 向量化
- 编译层：-O3 -ffast-math -march=armv8-a

研讨要求完成：

- ① 4 接口：iopointer / ioself_profiling / ioself / iofile
- ② 纳秒级时间函数：timestamp() / timestamp_diff()
- ③ 测试主函数循环多种规模，采样独立
- ④ 6+ 源文件：h/c 各 2 + test + demo + readme